

SUMMER INTERNSHIP REPORT

Networking and Cyber Security

Information Security Education and Awareness (ISEA) Phase III

Department of Computer Science and Engineering

Tezpur University

Multi-Client TCP Chat Application

with Graphical User Interface

Assignment 6 — GUI-Based Multi-Client Chat Application Using TCP

A Tkinter-based graphical extension of a multithreaded TCP chat system, featuring real-time broadcast and private messaging, live user presence tracking, and packet-level verification using Wireshark inside a simulated Mininet topology.

Student Name	Parinita Devi
Roll No.	CS-BTC25-39
Assignment	Assignment 6 (GUI-Based Multi-Client Chat Application Using TCP)
Technology Stack	Python 3, TCP Sockets, Tkinter, Threading, Mininet, Wireshark

Submitted as part of Summer Internship / Course Assignment Requirements

1. Objective

The primary objective of Assignment 6 is to extend the multi-client TCP chat application developed in Assignment 5 by converting its terminal-based client interface into a graphical desktop application using Python. The backend server implementation is reused with minimal modifications to preserve core networking features such as TCP socket handling, thread allocation, message broadcasting, private message routing, online user tracking, persistent chat history logging, and real-time user join/leave notifications. The assignment emphasizes GUI programming, event-driven architecture, non-blocking multithreaded network execution, and packet-level verification using Wireshark inside a simulated Mininet network topology.

2. GUI Design

The client interface (`client_gui.py`) was developed using Python's native GUI modules (`tkinter`, `tkinter.ttk`, and `tkinter.scrolledtext`). It features a clean, responsive two-stage workflow designed for ease of use, as sketched below.

```
+-----+
|           Multi-Client TCP Chat Application           |
+-----+-----+
| User: h2_user | Status: Connected                    | [ Disconnect ] |
+-----+-----+
| SYSTEM: h2_user has joined the chat!                  | Online Users      |
| SYSTEM: h3_user has joined the chat!                  | -----          |
| h2_user: Hello everyone!                              | h2_user           |
| [Private to h3_user]: Secret message                  | h3_user           |
|                                                       |                   |
+-----+-----+
| [ Message Input Box ] | [ Send ]                    |
+-----+-----+
```

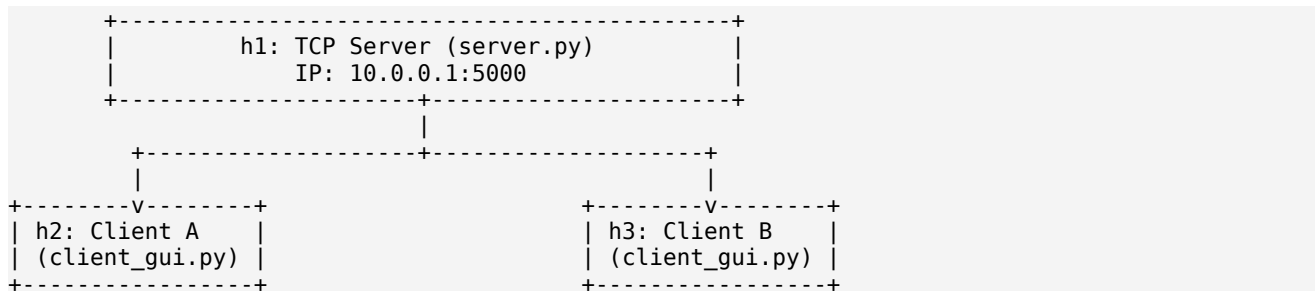
Key Interface Components & Layout Structure

- **Login Window / Connection Dialog (Task 1):** A modal dialog that prompts the user for the Server IP (e.g., 10.0.0.1), Port Number (5000), and a Username. It validates input to prevent empty usernames and handles connection failures gracefully before granting access to the main window.
- **Main Chat Display Area (Task 2):** A large, auto-scrolling text display constructed using `scrolledtext.ScrolledText`. It dynamically renders timestamped broadcast messages, direct messages, and system alerts.
- **Online User Panel (Task 3):** A dedicated sidebar panel using a `Listbox` / `Frame` widget that automatically refreshes to list active connected usernames (`h2_user`, `h3_user`, etc.).
- **Status Bar & Controls (Task 2):** Displays current user information (User: `h2_user` | Status: Connected) and includes a top-right Disconnect button to terminate the socket session cleanly.
- **Input Bar (Task 2):** Positioned at the bottom, containing an `Entry` box for message entry and a Send button. Pressing Enter or clicking Send transmits the message over TCP.

3. System Architecture

The application uses a Client-Server Multithreaded Architecture deployed across a 5-node Mininet topology (`sudo mn --topo single,5`):

- **Server (h1 - 10.0.0.1):** Hosts server.py listening on TCP port 5000.
- **Clients (h2, h3, h4, h5):** Host nodes running client_gui.py representing individual users (Client A, Client B, Client C, Client D).



Execution & Data Flow

- **Connection Initialization:** A client enters connection details into the GUI login window. The application connects a TCP socket (socket.AF_INET, socket.SOCK_STREAM) to 10.0.0.1:5000.
- **Server Registration:** The server accepts the socket connection (accept()), stores user state (username, IP, port, login time), and broadcasts a join alert to all connected sockets.
- **Threading Separation:** The GUI launches a dedicated background thread (threading.Thread) executing a socket receiver loop (recv()).
- **Message Routing:** Standard messages are sent to the server and relayed to all active client sockets (broadcast); direct messages formatted as /msg <username> <message> or @<username> <message> are parsed by the server and delivered exclusively to the target user socket (private message).
- **Teardown:** Clicking Disconnect sends a termination payload to the server, closes the TCP socket cleanly, broadcasts a leave notification, and updates user lists across active clients.

4. Components Reused from Assignment 5

Per assignment guidelines, core networking logic from Assignment 5 was preserved and reused rather than written from scratch:

- **TCP Socket Management:** Server socket binding (bind()), port listening (listen()), and incoming connection acceptance (accept()).
- **Multi-Client Thread Allocation:** Spawning a dedicated server worker thread per connected client to process concurrent client streams.
- **Message Routing Engine:** The server-side routing logic for handling broadcast messages and extracting private message destinations (/msg <username> <msg>).
- **Online User Tracking:** State management dictionary mapping usernames to active socket descriptors and client metadata.
- **Persistent History & Logging:** Server-side append-logging of all transactions to chat_history.csv with fields timestamp, sender, receiver, message_type, message.

5. GUI Design Decisions

- **Non-Blocking Multithreaded Architecture:** The primary design challenge in GUI-based networking is avoiding frozen windows. Socket reads (`socket.recv()`) are blocking calls. Running `recv()` inside a dedicated background thread ensures the main Tkinter GUI event loop (`mainloop()`) remains responsive, allowing users to type and click continuously.
- **Event-Driven Messaging:** Buttons and keypresses (e.g., Return key binding) trigger callback functions (`send_message()`), which extract text from entry fields, format the network packet, transmit it asynchronously, and clear the input box.
- **Separation of Concerns:** The network layer (socket handling) is separated from the interface presentation layer (tkinter widgets). The socket thread interacts with the GUI by updating widget contents safely, ensuring clean code maintainability.

6. Testing Results

System functionality was evaluated across multiple Mininet client windows (h2_user, h3_user). Execution output was captured and saved into the screenshots/ directory.

6.1 Login / Connection Dialog

The GUI startup login dialog prompts for the Server IP (10.0.0.1), Port (5000), and Username before establishing the TCP connection.

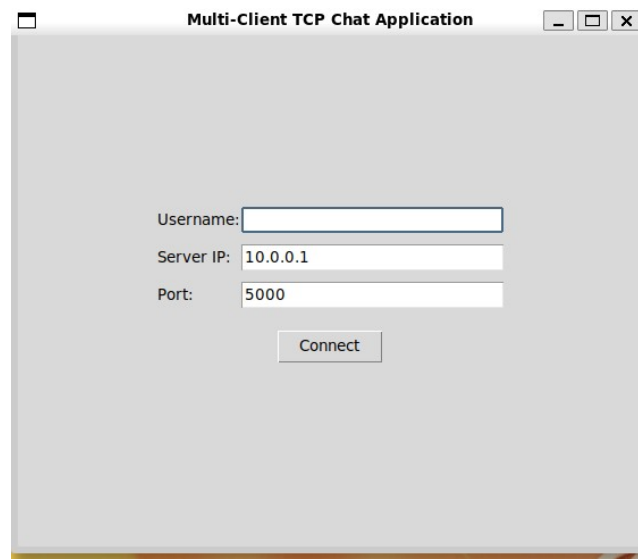


Figure 1: GUI login / connection dialog.

6.2 Broadcast Messaging

The connected client window shows join notifications and a broadcast message ("Hello everyone!") sent to all active participants.

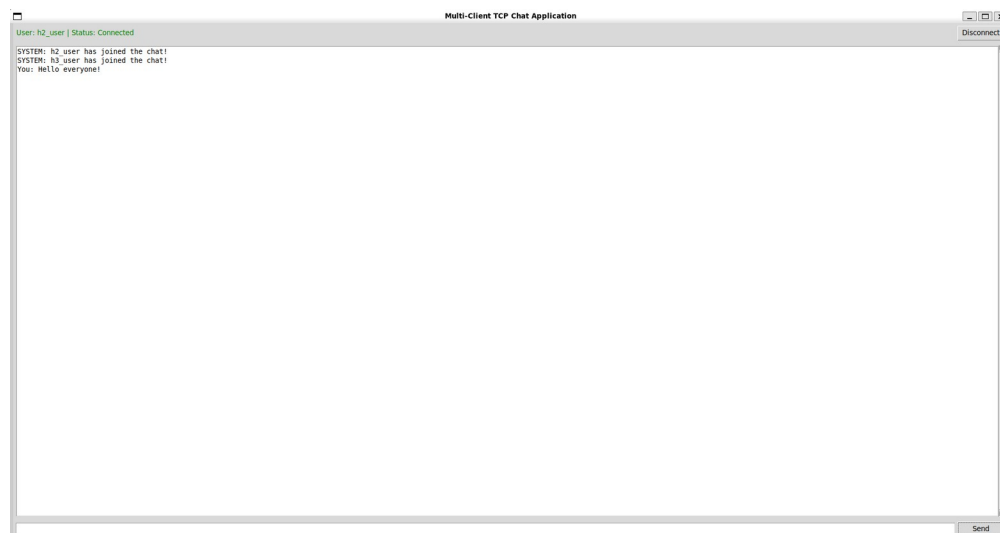


Figure 2: Broadcast message delivery confirmed in the chat window.

6.3 Private Messaging

Side-by-side client windows (h2_user and h3_user) confirm that a direct message sent via /msg h3_user (or @h3_user) is delivered exclusively to the intended recipient, appearing as "[Private to h3_user]" and "[Private from h2_user]" respectively.

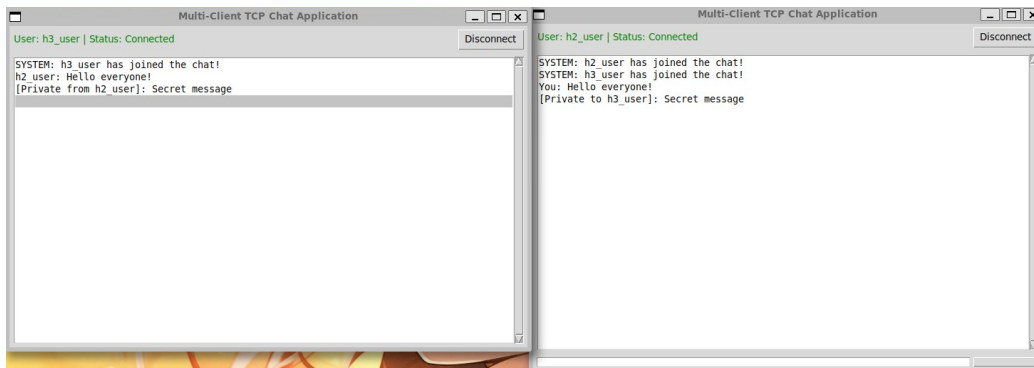


Figure 3: Private message exchange verified across two client windows.

6.4 User Join Notification

A real-time system notification ("SYSTEM: h3_user has joined the chat!") is broadcast to all connected clients the moment a new user logs in.

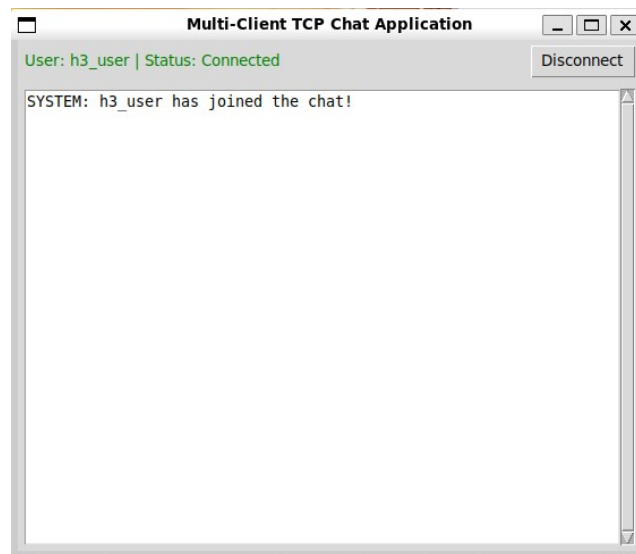


Figure 4: Real-time user join notification.

6.5 User Leave Notification

Clicking Disconnect triggers a system leave notification ("SYSTEM: h3_user has left the chat!"), which is broadcast to all remaining connected clients.

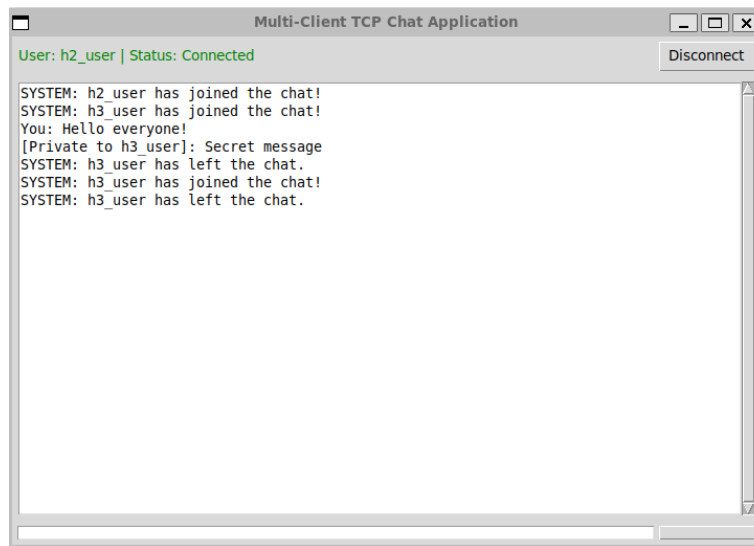


Figure 5: Real-time user leave notification after disconnection.

7. Wireshark Verification

Packet traces were captured on interface s1-eth1 using display filter tcp.port == 5000 to verify protocol behavior.

7.1 Client Connection

Captures the 3-way TCP handshake sequence (SYN, SYN-ACK, ACK packets) established between a client and the server at 10.0.0.1:5000.

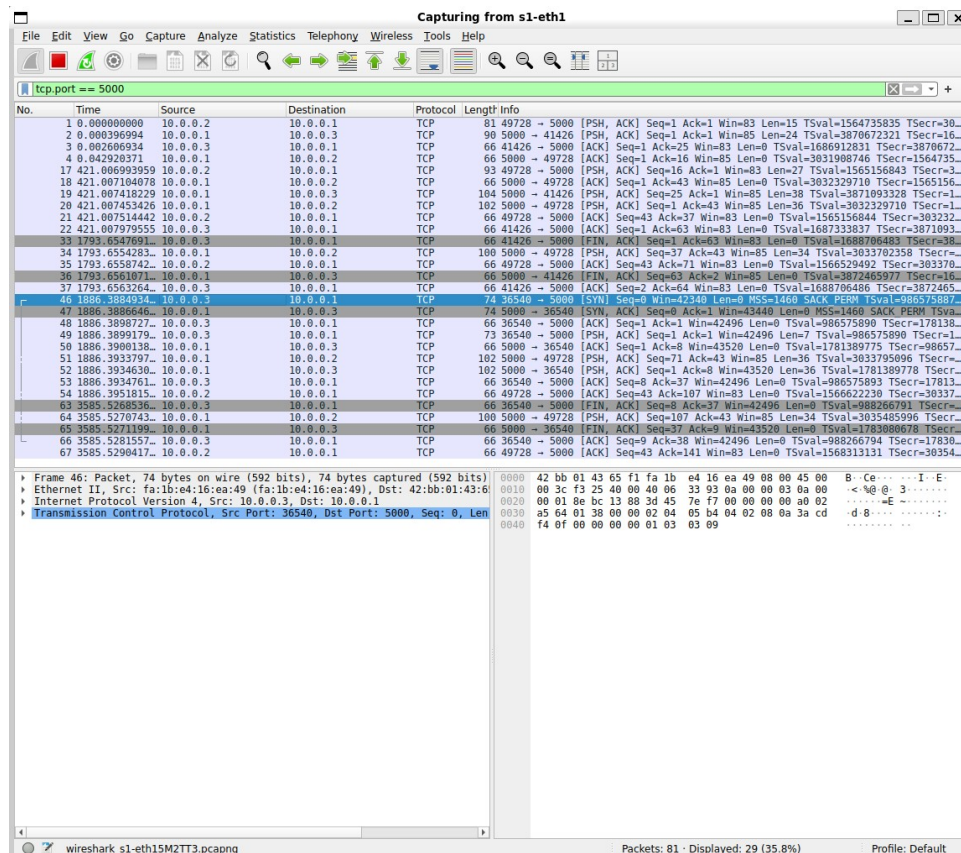


Figure 6: TCP three-way handshake for client connection.

7.2 Broadcast Payload

Captures data packets with the PSH, ACK flags. ASCII inspection in the packet detail pane confirms the clear-text payload "Hello everyone!" being relayed from the sending client through the server to another connected client.

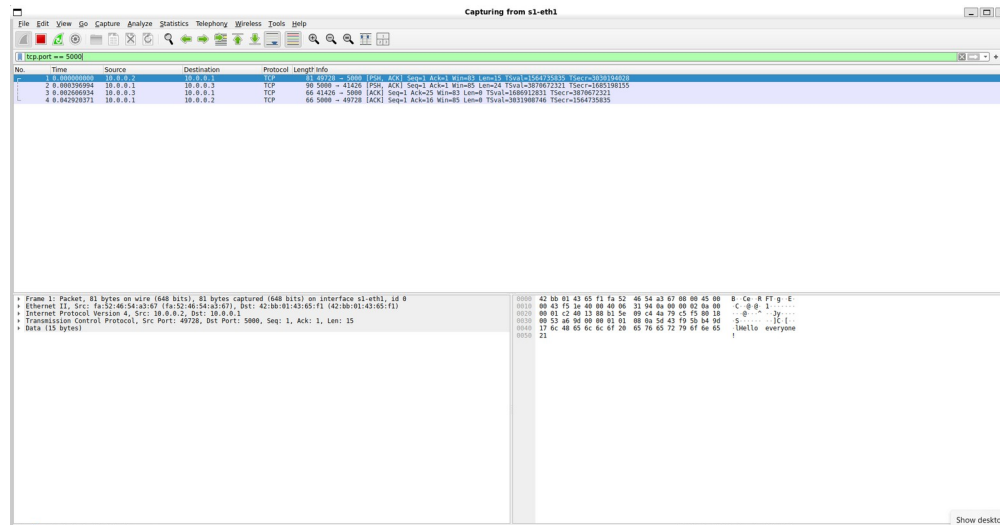


Figure 7: Broadcast payload captured with clear-text message content.

7.3 Private Message Payload

Identifies directed PSH, ACK data frames carrying private text strings (/msg h3_user Secret message) targeted exclusively to a single destination socket.

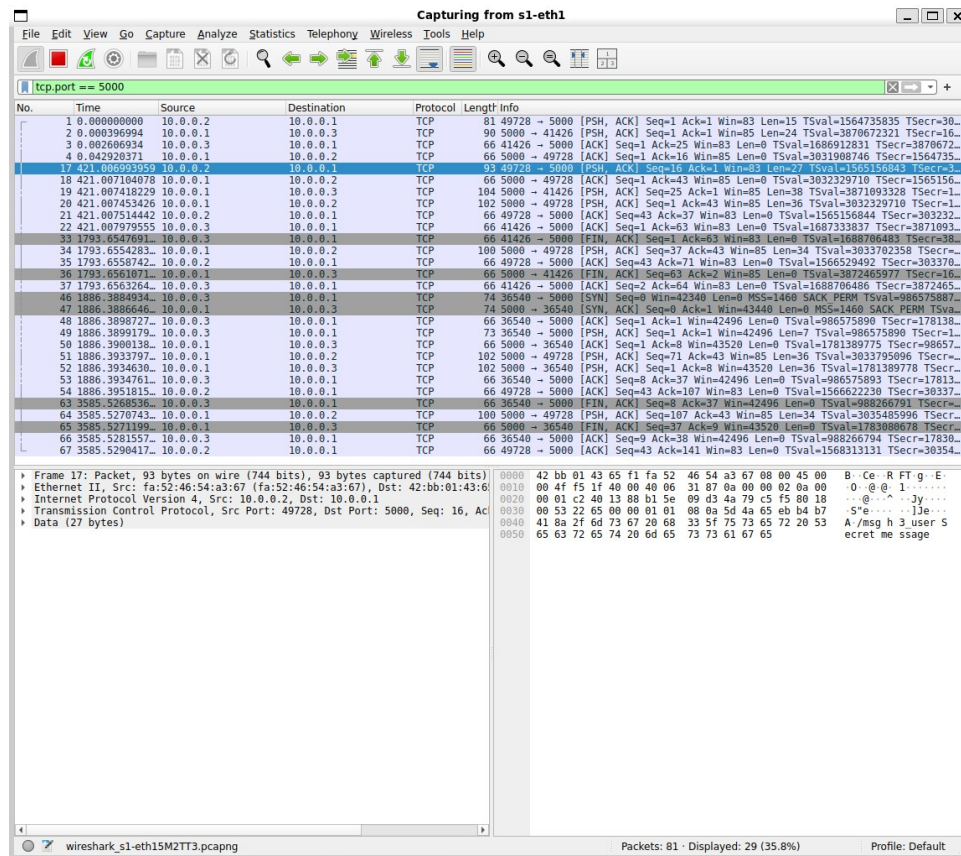


Figure 8: Private message payload captured at the packet level.

7.4 Client Disconnection

Validates the formal TCP connection teardown sequence (FIN, ACK and ACK frames) initiated when the Disconnect button is pressed.

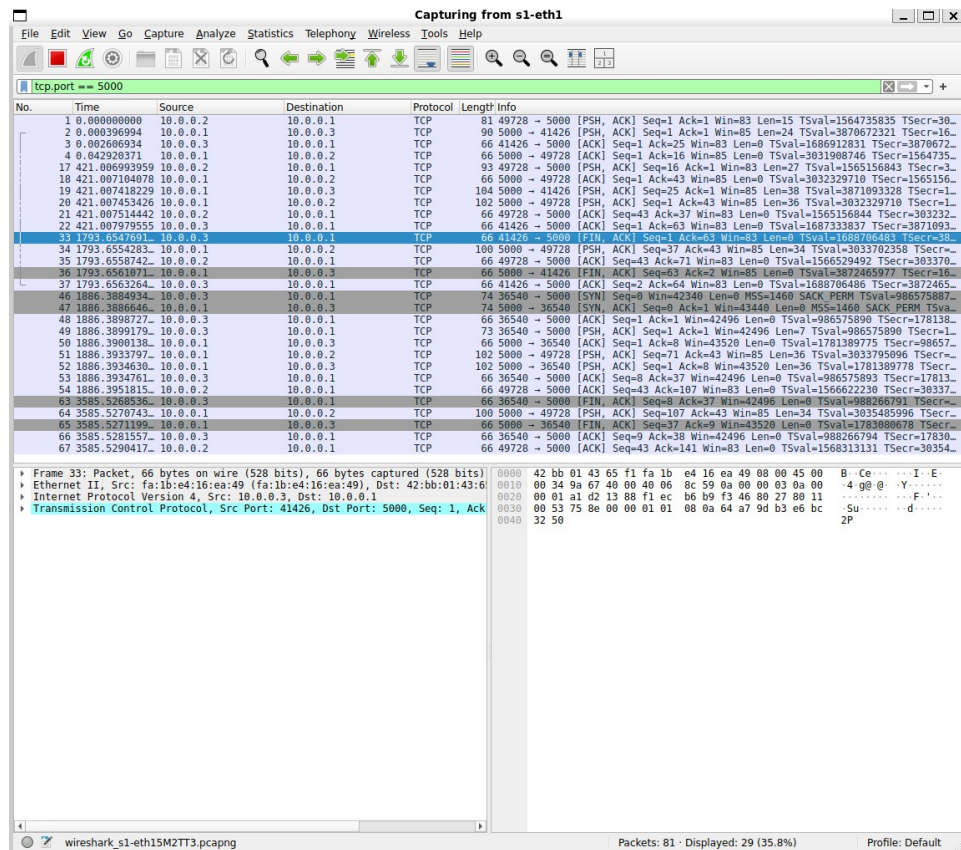


Figure 9: TCP connection teardown on client disconnection.

8. Reflection Answers

Q1. Why should networking logic and GUI code be separated?

Separating networking logic from GUI code promotes modularity, maintainability, and code reusability. By keeping socket operations independent of widget rendering, developers can update or replace the interface (e.g., switching from Tkinter to Qt or a web front-end) without modifying underlying network socket protocols.

Q2. Why is a background thread required?

Socket reading functions like `socket.recv()` are blocking calls that stall code execution until data arrives from the network. If run on the main thread, `recv()` would block Tkinter's event loop (`mainloop()`), causing the GUI window to freeze and stop responding to mouse clicks or keyboard input. Spawning a background receiver thread isolates socket waiting operations, leaving the GUI thread responsive.

Q3. Which components from Assignment 5 were reused?

Reused components include TCP socket management (`bind`, `listen`, `accept`), the multi-client server threading model, server-side message routing mechanisms (broadcast and private /msg parsing), client state tracking structures, and persistent CSV chat history logging.

Q4. How did the GUI improve usability?

The GUI replaced abstract terminal prompts with visual controls. Users benefit from a continuous chat display, a live sidebar showing online participants, one-click action buttons, auto-scrolling message views, and clear connection status headers.

Q5. Suggest one future enhancement.

Implement Transport Layer Security (TLS/SSL) encryption over the TCP sockets. Encrypting payload streams will prevent plain-text inspection of private chat messages in packet capture tools like Wireshark, providing secure communication.

9. Conclusion

Assignment 6 successfully transformed the terminal-based TCP chat system into a responsive desktop application. By integrating Python's tkinter interface with non-blocking background network threads, the system provides real-time multi-user communication, user presence tracking, and private messaging. The application was verified inside Mininet across multiple virtual nodes and validated at the packet level using Wireshark. All source files, documentation, and screenshots have been organized into the required submission package structure.